

Assignment 3: RAG for Science Question Answering

Student ID: 314832008

Source Code: <https://github.com/tingul4/RNN-course>

1. Overview

In this assignment, I built a full Retrieval-Augmented Generation (RAG) pipeline to answer multiple-choice science questions from the Kaggle LLM Science Exam dataset. The pipeline covers three stages: (1) data chunking and indexing, (2) two-stage retrieval with vector search and cross-encoder re-ranking, and (3) answer generation using a local LLM.

Experimental Setup

The knowledge base was constructed directly from the provided `train.csv` dataset, which was downloaded from the official [Kaggle LLM Science Exam competition](#) and contains 200 science-related multiple-choice questions. Each question has five answer choices (A–E). To facilitate effective retrieval, I transformed the dataset by treating each `(question_prompt + one answer option)` pair as a separate, individual document. This transformation creates a corpus of **1,000 documents** covering diverse STEM topics. By querying the system with the pure question prompt, we can objectively measure the retrieval quality (Recall@3) by checking whether the document containing the correct answer option is successfully retrieved.

Models used:

- Embedding: `BAAI/bge-m3` (CUDA)
 - Re-ranker: `cross-encoder/ms-marco-MiniLM-L-6-v2` (CUDA)
 - Generator: `llama3:8b` (via Ollama)
-

2. Part 1 — The Indexing Pipeline

2.1 Chunking Strategies

I implemented two distinct chunking strategies:

- **Method A (Fixed-Size Chunking):** Used naive fixed-size chunking via `RecursiveCharacterTextSplitter`. I used `chunk_size=500` with a standard `chunk_overlap=50`. This establishes a baseline for retrieving raw data block by block based on character count.
 - **Method B (Semantic Chunking):** Utilized LangChain's `SemanticChunker` with the `BAAI/bge-m3` embedding model. Instead of splitting by character count, this method dynamically splits text based on semantic similarity between sentences, aiming to keep related concepts together and break at topic shifts.
-

3. Part 2 — Retrieval & Re-ranking

3.1 Setup

- **Stage 1 (Dense Retrieval):** Vector similarity search with `k=20` candidates.
- **Stage 2 (Re-ranking):** Cross-encoder scores each (query, candidate) pair; top-3 are kept as the final context.
- **Baseline:** Vector-only retrieval returning top-3 directly (no cross-encoder).
- **Metric:** Recall@3 — whether the correct answer option appears in the top-3 results.

Experiments were run on the first 50 training questions.

3.2 Hit Rate Results

Configuration	Index A (Fixed, size=500)	Index B (Semantic Chunker)
Vector Search Only	0.60	0.58
Vector Search + Rerank	0.70	0.62
Improvement	+0.10 (+10%)	+0.04 (+4%)

(Note: Unlike long-form documents, the Kaggle dataset consists of short, self-contained Q/A pairs. The `SemanticChunker` (Index B) dynamically splits text based on sentence-level semantic shifts, which occasionally over-fragments a single long answer option and breaks its context. On the other hand, the Fixed chunking strategy (Index A) safely wraps the short Q/A pairs within its character limit, preserving their integrity. Consequently, naive fixed chunking yielded slightly better retrieval performance for this specific dataset.)

4. Part 3 — Generation with Local LLM

4.1 Setup

- **LLM Integration:** Connected the end-to-end pipeline to a local LLM API server running `llama3:8b` via Ollama. The top-3 context chunks from Stage 2 re-ranking were concatenated and fed into the prompt as context.
- **System Prompt Design:** The prompt includes the retrieved context, the question, and all five answer options (A–E). The LLM is instructed:

"You are a helpful science assistant. Use the context below to answer the multiple-choice question. If the correct answer cannot be deduced from the context, output 'F' meaning 'I do not know'. Reply with ONLY the letter of the correct answer (A, B, C, D, or E). No explanation."

Unlike a free-form generation approach, presenting the MCQ options directly allows the LLM to reason over all candidates and output a single letter, which is parsed by regex. This also makes the evaluation deterministic and avoids the noise introduced by cosine-similarity-based option matching.

The strict "output F if unknown" instruction is intentional: it forces the model to refrain from guessing when the context provides no relevant signal. Outputs of 'F' (or any non-A–E response) are counted as incorrect, incentivizing the model to use context when available rather than hallucinating an answer.

- **Baseline (No RAG):** To isolate the effect of retrieval, a second evaluation was run where the LLM received only the question and the five answer options—no retrieved context—and was asked to answer from its own parametric knowledge.

4.2 Accuracy Results

Evaluated on the first 50 questions from `train.csv`.

Configuration	Accuracy
Without RAG	0.66
With RAG	0.54

The RAG pipeline underperformed the no-RAG baseline by **12 percentage points**. This counterintuitive result is explained by a fundamental mismatch between the corpus design and the cross-encoder's scoring objective:

1. **Corpus is MCQ options, not knowledge paragraphs.** The vector database was built from (question + one answer option) pairs derived from `train.csv`. When a question is queried, the most semantically similar documents are the other answer options for the *same question*—not external explanatory knowledge. These documents do not state *which* option is correct.
2. **Cross-encoder promotes plausible-sounding wrong answers.** The re-ranker (`ms-marco-MiniLM-L-6-v2`) was trained on web passage relevance, not MCQ answer selection. It scores how *relevant* a passage sounds to a query, not whether it is factually correct. Distractor options are deliberately crafted to appear plausible, so they often receive high relevance scores. For example, on ID#0 (MOND galaxy clusters, correct = D), the re-ranker ranked Option B—a wrong but superficially similar answer—at the top, and the LLM followed the misleading context.
3. **LLM is anchored to the provided context.** When the retrieved context highlights a wrong option, the LLM's generation is biased toward that option even when its own parametric knowledge would have answered correctly. Without RAG, `llama3:8b` answers from pretraining knowledge and achieves 66% accuracy on these STEM questions.

Note on corpus limitation: The assignment specification calls for a Wikipedia plain-text corpus as the knowledge base. Because only `train.csv` was available (the Wikipedia parquet file was not downloaded), the RAG corpus was substituted with the Q&A pairs from `train.csv`. This substitution is the root cause of the underperformance and is an important finding: **RAG improves accuracy only when the corpus provides genuine external knowledge that the LLM does not already possess—not when the corpus mirrors the format of the questions themselves.**

5. Discussion

5.1 Chunk Size Analysis

I evaluated the impact of different chunk sizes by comparing `chunk_size=500` and `chunk_size=1000` using the Fixed chunking strategy (with Stage 1 Vector Search only) on the query: *"What generates energy in the cell?"*

- **Chunk Size = 500 (Score ~0.85):** The system retrieved chunks related to "power density in energy systems" and "renewable energy." It matched the keyword "energy" but completely missed the biological context ("in the cell"). Furthermore, all top three retrieved chunks were identical. This indicates that a smaller chunk size over-fragments the text, likely severing the question prompt from its corresponding answer choices, resulting in redundant and context-poor semantics.
- **Chunk Size = 1000 (Score ~0.92-0.93):** The system accurately retrieved relevant documents discussing "active and passive transport in cells" and "energy input from the cell." The larger chunk size successfully encapsulated the entire question prompt alongside its options within the same chunk. This preserved the crucial semantic relationship between "energy" and "cells," yielding highly relevant hits.

Conclusion: A `chunk_size` of 1000 is significantly more effective for this dataset. Smaller chunks (e.g., 500 characters) suffer from context loss and fragmentation, causing the embedding model to focus on isolated keywords (like "energy" in physics) rather than the complete conceptual meaning.

5.2 Re-ranking Impact

The re-ranking stage plays a critical role in overcoming the limitations of standard vector search, especially for complex scientific queries where exact conceptual relationships matter more than keyword overlap. Here are two examples demonstrating this:

Example 1 — ID#3 (What is the significance of regularization in terms of renormalization problems in physics?, correct = C)

- **Stage 1 (Vector Search):** The dense vector search retrieved Options E, A, and B in the top-3 — all plausible-sounding candidates using similar terminology ("finite radius," "infinite radius," "mass-energy of an electron"), but none of them was the correct answer. Option C, which correctly describes regularization as a means of demonstrating computability of systems below a certain size, was ranked outside the top-3 by the embedding model.
- **Stage 2 (Re-ranking):** The cross-encoder processes each (query, candidate) pair jointly rather than independently. It correctly distinguished the subtle semantic difference between Option C and the other distractors, successfully promoting Option C (score: 9.50) to the #1 position and demoting the less precise options.

Example 2 — ID#11 (What is the relationship between the Wigner function and the density matrix operator?, correct = A)

- **Stage 1 (Vector Search):** The vector search retrieved several irrelevant documents that mentioned "Wigner function" or "density matrix" in isolation, leaving the correct answer out of the top results. It struggled to evaluate the nuanced relationship being asked for.
- **Stage 2 (Re-ranking):** The cross-encoder effectively interpreted the relational aspect of the query ("What is the relationship between..."). It scanned the initial top-20 candidates and bumped the correct document (Option A) to the #1 position because it explicitly explained the mathematical mapping between the two concepts rather than just randomly hitting the keywords.

5.3 Latency

Measured across 50 queries on an RTX 4090:

Stage	Typical Latency	Notes
Vector Search	~0.035–0.050 s	First query ~0.45 s (index cold-start); subsequent queries warm
Re-ranking (Cross-Encoder)	~0.032–0.040 s	Scores 20 (query, doc) pairs via GPU; fast due to small batch
LLM Generation	~0.43 s (warm avg)	First call ~12.5 s (Ollama model load); subsequent calls stable

- **Vector Search** is the fastest stage after warm-up, leveraging approximate nearest-neighbor search in ChromaDB.
- **Re-ranking** adds only ~35 ms on average, well within acceptable latency for most applications.
- **LLM Generation** dominates total latency at ~0.43 s per query, as each forward pass decodes the response token-by-token.
- **Is the re-ranking worth the extra time cost?** From a **retrieval quality** perspective, yes: re-ranking improved the Recall@3 hit rate by +10% on Index A (Fixed-500) at a cost of only ~35 ms, which is less than 10% of the total pipeline latency. The cross-encoder adds high-precision scoring at negligible cost relative to LLM generation.

However, from an **end-to-end accuracy** perspective in this specific experiment, the benefit was not realized because the corpus itself was the limiting factor. When retrieval context misleads the LLM (as analyzed in Section 4.2), improving retrieval precision alone does not improve final accuracy. With a proper knowledge corpus (e.g., Wikipedia passages), the re-ranking investment would translate directly into accuracy gains.